

Deep Learning y Sistemas Inteligentes - Laboratorio 7

- Josue Marroquin 22397
- Sebastian Huertas 22295

Referencias:

- Box2D -> <https://pypi.org/project/Box2D/>
- Cart Pole -> https://gymnasium.farama.org/environments/classic_control/cart_pole/

1) Librerias

```
In [2]: import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import random
from collections import deque
import matplotlib.pyplot as plt
```

2) Entorno

```
In [3]: import gymnasium as gym
import numpy as np

# Crear el entorno CartPole-v1
env = gym.make("CartPole-v1")

# Información del entorno
print("Action Space:", env.action_space)
print("Observation Space:", env.observation_space)
print("Observation Space Shape:", env.observation_space.shape)
print("Observation Space High:", env.observation_space.high)
print("Observation Space Low:", env.observation_space.low)
```

```
Action Space: Discrete(2)
Observation Space: Box([-4.8          -inf -0.41887903        -inf], [4.8
inf 0.41887903          inf], (4,), float32)
Observation Space Shape: (4,)
Observation Space High: [4.8          inf 0.41887903        inf]
Observation Space Low: [-4.8          -inf -0.41887903        -inf]
```

3) Redes en linea y de destino

```
In [4]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Usando dispositivo: {device}")
```

Usando dispositivo: cpu

```
In [5]: class DQN(nn.Module):
        """Red neuronal para DQN"""
        def __init__(self, state_size=4, action_size=2, hidden_size=128):
            super(DQN, self).__init__()
            self.fc1 = nn.Linear(state_size, hidden_size)
            self.fc2 = nn.Linear(hidden_size, hidden_size)
            self.fc3 = nn.Linear(hidden_size, action_size)
            self.relu = nn.ReLU()

        def forward(self, x):
            x = self.relu(self.fc1(x))
            x = self.relu(self.fc2(x))
            x = self.fc3(x)
            return x

        # Crear las redes
        state_size = env.observation_space.shape[0] # 4 (posición cart, velocidad cart, ángulo cart, velocidad angular)
        action_size = env.action_space.n # 2 (izquierda, derecha)

        # Red en línea (online network) - se actualiza frecuentemente
        q_network = DQN(state_size, action_size).to(device)

        # Red de destino (target network) - se actualiza periódicamente
        target_network = DQN(state_size, action_size).to(device)

        # Inicializar la red de destino con los mismos pesos que la red en línea
        target_network.load_state_dict(q_network.state_dict())

        # Optimizador solo para la red en línea
        optimizer = optim.Adam(q_network.parameters(), lr=0.001)

        print("Redes creadas:")
        print(f"Red en línea: {q_network}")
        print(f"Red de destino: {target_network}")
        print(f"Parámetros de la red: {sum(p.numel() for p in q_network.parameters())}")
```

Redes creadas:

Red en línea: DQN(

```
(fc1): Linear(in_features=4, out_features=128, bias=True)
(fc2): Linear(in_features=128, out_features=128, bias=True)
(fc3): Linear(in_features=128, out_features=2, bias=True)
(rel): ReLU()
)
```

Red de destino: DQN(

```
(fc1): Linear(in_features=4, out_features=128, bias=True)
(fc2): Linear(in_features=128, out_features=128, bias=True)
(fc3): Linear(in_features=128, out_features=2, bias=True)
(rel): ReLU()
)
```

Parámetros de la red: 17410

4) Establecer Hiperparametros

```
In [6]: NUM_EPISODES = 1000      # Número de episodios de entrenamiento
        MAX_STEPS = 500         # Máximo número de pasos por episodio
        BATCH_SIZE = 32        # Tamaño del batch para el entrenamiento
        LEARNING_RATE = 0.001  # Tasa de aprendizaje (ya definida en optimizer)

        # Parámetros del algoritmo DQN
        GAMMA = 0.99           # Factor de descuento (discount factor)
        TARGET_UPDATE = 10     # Frecuencia de actualización de la red objetivo (cada

        # Parámetros de exploración (epsilon-greedy)
        EPSILON_START = 1.0    # Epsilon inicial (100% exploración)
        EPSILON_END = 0.01     # Epsilon mínimo (1% exploración)
        EPSILON_DECAY = 0.995  # Factor de decaimiento del epsilon

        # Parámetros del Experience Replay Buffer
        BUFFER_SIZE = 10000    # Tamaño del buffer de experiencias
        MIN_BUFFER_SIZE = 1000 # Tamaño mínimo del buffer antes de entrenar

        # Variables de estado
        epsilon = EPSILON_START # Epsilon actual
        episode_rewards = []     # Lista para almacenar recompensas por episodio
        episode_lengths = []     # Lista para almacenar longitud de episodios
```

```
In [7]: # Función para calcular epsilon con decaimiento
def get_epsilon(episode):
    """Calcula el valor de epsilon para un episodio dado"""
    return max(EPSILON_END, EPSILON_START * (EPSILON_DECAY ** episode))

# Función para mostrar progreso del entrenamiento
def print_training_progress(episode, epsilon, reward, avg_reward):
    """Imprime el progreso del entrenamiento"""
    if episode % 100 == 0 or episode < 10:
        print(f"Episodio {episode:4d} | Epsilon: {epsilon:.3f} | Recompensa: {reward}")

# Función para evaluar el rendimiento
def evaluate_agent(num_episodes=10):
    """Evalúa el agente sin exploración (epsilon=0)"""
    total_reward = 0
    for _ in range(num_episodes):
        state, _ = env.reset()
        episode_reward = 0
        done = False

        while not done:
            # Selección de acción sin exploración
            with torch.no_grad():
                state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
                q_values = q_network(state_tensor)
                action = q_values.argmax().item()

            state, reward, terminated, truncated, _ = env.step(action)
            episode_reward += reward
            done = terminated or truncated

        total_reward += episode_reward
```

```
return total_reward / num_episodes
```

5) Selección de acciones epsilon-greedy

```
In [8]: def select_action(state, epsilon):
        """
        Selecciona una acción usando la política epsilon-greedy

        Args:
            state: Estado actual del entorno
            epsilon: Probabilidad de exploración (0-1)

        Returns:
            action: Acción seleccionada (0 o 1)
        """
        if random.random() < epsilon:
            # Exploración: seleccionar acción aleatoria
            return env.action_space.sample()
        else:
            # Explotación: seleccionar la mejor acción según Q-network
            with torch.no_grad():
                state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
                q_values = q_network(state_tensor)
                return q_values.argmax().item()

    def update_target_network():
        """
        Actualiza la red de destino copiando los pesos de la red en línea
        """
        target_network.load_state_dict(q_network.state_dict())

    # Probar la función de selección de acciones
    print("Probando selección de acciones epsilon-greedy:")
    test_state, _ = env.reset()
    print(f"Estado de prueba: {test_state}")

    # Probar con diferentes valores de epsilon
    epsilons = [1.0, 0.5, 0.1, 0.0]
    for eps in epsilons:
        actions = []
        for _ in range(10):
            action = select_action(test_state, eps)
            actions.append(action)

        exploration_rate = sum(actions) / len(actions) if len(actions) > 0 else 0
        print(f"Epsilon {eps:.1f}: Acciones = {actions[:5]}... (Tasa exploración aprox:
```

Probando selección de acciones epsilon-greedy:

Estado de prueba: [-0.03602611 0.04447041 -0.03204983 0.01885743]

Epsilon 1.0: Acciones = [np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(1)]... (Tasa exploración aprox: 0.60)

Epsilon 0.5: Acciones = [np.int64(1), 0, 0, 0, np.int64(0)]... (Tasa exploración aprox: 0.10)

Epsilon 0.1: Acciones = [0, 0, 0, np.int64(1), 0]... (Tasa exploración aprox: 0.10)

Epsilon 0.0: Acciones = [0, 0, 0, 0, 0]... (Tasa exploración aprox: 0.00)

6) Reproducción de la experiencia

```
In [9]: class ReplayBuffer:
    """
    Buffer circular para almacenar experiencias del agente
    """

    def __init__(self, capacity=10000):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        """
        Añade una experiencia al buffer

        Args:
            state: Estado actual
            action: Acción tomada
            reward: Recompensa recibida
            next_state: Siguiendo estado
            done: Si el episodio terminó
        """
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        """
        Muestra un batch aleatorio de experiencias

        Args:
            batch_size: Tamaño del batch a muestrear

        Returns:
            Tuple de tensores (states, actions, rewards, next_states, dones)
        """
        batch = random.sample(self.buffer, batch_size)

        # Separar los componentes del batch
        states = torch.FloatTensor([e[0] for e in batch]).to(device)
        actions = torch.LongTensor([e[1] for e in batch]).to(device)
        rewards = torch.FloatTensor([e[2] for e in batch]).to(device)
        next_states = torch.FloatTensor([e[3] for e in batch]).to(device)
        dones = torch.BoolTensor([e[4] for e in batch]).to(device)

        return states, actions, rewards, next_states, dones

    def __len__(self):
        return len(self.buffer)
```

```
# Crear el buffer de experiencias
replay_buffer = ReplayBuffer(BUFFER_SIZE)

print(f"Buffer de experiencias creado con capacidad: {BUFFER_SIZE}")
print(f"Tamaño mínimo antes de entrenar: {MIN_BUFFER_SIZE}")
```

Buffer de experiencias creado con capacidad: 10000

Tamaño mínimo antes de entrenar: 1000

7/8) Ciclo de entrenamiento y visualización

```
In [10]: def train_dqn():
    """
    Entrena la red DQN usando experience replay
    """
    if len(replay_buffer) < MIN_BUFFER_SIZE:
        return None

    # Muestrear batch del buffer
    states, actions, rewards, next_states, dones = replay_buffer.sample(BATCH_SIZE)

    # Calcular Q-values actuales
    current_q_values = q_network(states).gather(1, actions.unsqueeze(1))

    # Calcular Q-values objetivo usando la red de destino
    with torch.no_grad():
        next_q_values = target_network(next_states).max(1)[0]
        target_q_values = rewards + (GAMMA * next_q_values * ~dones)

    # Calcular pérdida
    loss = nn.MSELoss()(current_q_values.squeeze(), target_q_values)

    # Actualizar la red
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    return loss.item()

# Función principal de entrenamiento
def train_agent(render=0):
    """
    Ciclo principal de entrenamiento del agente DQN

    Args:
        render: 1 para renderizar (con visualización), 0 para no renderizar (más rá
    """
    episode_rewards = []
    episode_losses = []

    # Crear entorno apropiado según el parámetro render
    if render == 1:
        env_training = gym.make("CartPole-v1", render_mode="human")
        print("Iniciando entrenamiento DQN con visualización...")
    else:
```

```

env_training = env # Usar el entorno sin renderizado
print("Iniciando entrenamiento DQN...")

print("=" * 60)

for episode in range(NUM_EPISODES):
    # Resetear el entorno
    state, _ = env_training.reset()
    episode_reward = 0
    episode_loss = []

    # Calcular epsilon para este episodio
    current_epsilon = get_epsilon(episode)

    for step in range(MAX_STEPS):
        # Seleccionar acción usando epsilon-greedy
        action = select_action(state, current_epsilon)

        # Ejecutar acción en el entorno
        next_state, reward, terminated, truncated, _ = env_training.step(action)
        done = terminated or truncated

        # Almacenar experiencia en el buffer
        replay_buffer.push(state, action, reward, next_state, done)

        # Entrenar la red si hay suficientes experiencias
        loss = train_dqn()
        if loss is not None:
            episode_loss.append(loss)

        # Actualizar estado y recompensa acumulada
        state = next_state
        episode_reward += reward

        # Si el episodio terminó, salir del loop
        if done:
            break

    # Actualizar la red de destino cada TARGET_UPDATE episodios
    if episode % TARGET_UPDATE == 0:
        update_target_network()
        print(f"Red de destino actualizada en episodio {episode}")

    # Registrar métricas del episodio
    episode_rewards.append(episode_reward)
    avg_loss = np.mean(episode_loss) if episode_loss else 0
    episode_losses.append(avg_loss)

    # Mostrar progreso
    if len(episode_rewards) >= 100:
        avg_reward = np.mean(episode_rewards[-100:])
    else:
        avg_reward = np.mean(episode_rewards)

    print_training_progress(episode, current_epsilon, episode_reward, avg_reward)

```

```

# Evaluación periódica
if episode % 200 == 0 and episode > 0:
    eval_reward = evaluate_agent(10)
    print(f"Evaluación episodio {episode}: Recompensa promedio = {eval_rewa

# Criterio de parada temprana (opcional)
if len(episode_rewards) >= 100 and avg_reward >= 475:
    print(f"\n¡Problema resuelto en episodio {episode}!")
    print(f"Recompensa promedio últimos 100 episodios: {avg_reward:.2f}")
    break

# Cerrar entorno de entrenamiento si se creó uno específico
if render == 1:
    env_training.close()

print("\nEntrenamiento completado!")
return episode_rewards, episode_losses

```

```

In [11]: # Ejecucion del entrenamiento

print("¡Iniciando entrenamiento del agente DQN!")
print(f"Configuración:")
print(f"- Episodios: {NUM_EPISODES}")
print(f"- Epsilon inicial: {EPSILON_START}")
print(f"- Epsilon final: {EPSILON_END}")
print(f"- Actualización de red objetivo cada: {TARGET_UPDATE} episodios")
print(f"- Tamaño del buffer: {BUFFER_SIZE}")

# Ejecutar entrenamiento
rewards, losses = train_agent(render=0) # Cambiar a 1 para ver la visualización, no

print(f"\nResultados del entrenamiento:")
print(f"- Episodios completados: {len(rewards)}")
print(f"- Recompensa promedio últimos 100 episodios: {np.mean(rewards[-100:]):.2f}")
print(f"- Recompensa máxima: {max(rewards):.2f}")
print(f"- Pérdida promedio: {np.mean([l for l in losses if l > 0]):.4f}")

```

¡Iniciando entrenamiento del agente DQN!

Configuración:

- Episodios: 1000
- Epsilon inicial: 1.0
- Epsilon final: 0.01
- Actualización de red objetivo cada: 10 episodios
- Tamaño del buffer: 10000

Iniciando entrenamiento DQN...

=====

Red de destino actualizada en episodio 0

Episodio	0	Epsilon: 1.000	Recompensa: 16	Promedio: 16.00
Episodio	1	Epsilon: 0.995	Recompensa: 21	Promedio: 18.50
Episodio	2	Epsilon: 0.990	Recompensa: 34	Promedio: 23.67
Episodio	3	Epsilon: 0.985	Recompensa: 18	Promedio: 22.25
Episodio	4	Epsilon: 0.980	Recompensa: 45	Promedio: 26.80
Episodio	5	Epsilon: 0.975	Recompensa: 24	Promedio: 26.33
Episodio	6	Epsilon: 0.970	Recompensa: 12	Promedio: 24.29
Episodio	7	Epsilon: 0.966	Recompensa: 18	Promedio: 23.50
Episodio	8	Epsilon: 0.961	Recompensa: 23	Promedio: 23.44
Episodio	9	Epsilon: 0.956	Recompensa: 23	Promedio: 23.40

Red de destino actualizada en episodio 10

Red de destino actualizada en episodio 20

Red de destino actualizada en episodio 30

Red de destino actualizada en episodio 40

C:\Users\thiag\AppData\Local\Temp\ipykernel_28564\3305122280.py:34: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. Please consider converting the list to a single numpy.ndarray with numpy.array() before converting to a tensor. (Triggered internally at C:\actions-runner_work\pytorch\pytorch\pytorch\torch\src\utils\tensor_new.cpp:256.)

states = torch.FloatTensor([e[0] for e in batch]).to(device)

Red de destino actualizada en episodio 50
Red de destino actualizada en episodio 60
Red de destino actualizada en episodio 70
Red de destino actualizada en episodio 80
Red de destino actualizada en episodio 90
Red de destino actualizada en episodio 100
Episodio 100 | Epsilon: 0.606 | Recompensa: 34 | Promedio: 24.01
Red de destino actualizada en episodio 110
Red de destino actualizada en episodio 120
Red de destino actualizada en episodio 130
Red de destino actualizada en episodio 140
Red de destino actualizada en episodio 150
Red de destino actualizada en episodio 160
Red de destino actualizada en episodio 170
Red de destino actualizada en episodio 180
Red de destino actualizada en episodio 190
Red de destino actualizada en episodio 200
Episodio 200 | Epsilon: 0.367 | Recompensa: 17 | Promedio: 88.16
Evaluación episodio 200: Recompensa promedio = 201.50
Red de destino actualizada en episodio 210
Red de destino actualizada en episodio 220
Red de destino actualizada en episodio 230
Red de destino actualizada en episodio 240
Red de destino actualizada en episodio 250
Red de destino actualizada en episodio 260
Red de destino actualizada en episodio 270
Red de destino actualizada en episodio 280
Red de destino actualizada en episodio 290
Red de destino actualizada en episodio 300
Episodio 300 | Epsilon: 0.222 | Recompensa: 44 | Promedio: 95.12
Red de destino actualizada en episodio 310
Red de destino actualizada en episodio 320
Red de destino actualizada en episodio 330
Red de destino actualizada en episodio 340
Red de destino actualizada en episodio 350
Red de destino actualizada en episodio 360
Red de destino actualizada en episodio 370
Red de destino actualizada en episodio 380
Red de destino actualizada en episodio 390
Red de destino actualizada en episodio 400
Episodio 400 | Epsilon: 0.135 | Recompensa: 35 | Promedio: 81.10
Evaluación episodio 400: Recompensa promedio = 193.10
Red de destino actualizada en episodio 410
Red de destino actualizada en episodio 420
Red de destino actualizada en episodio 430
Red de destino actualizada en episodio 440
Red de destino actualizada en episodio 450
Red de destino actualizada en episodio 460
Red de destino actualizada en episodio 470
Red de destino actualizada en episodio 480
Red de destino actualizada en episodio 490
Red de destino actualizada en episodio 500
Episodio 500 | Epsilon: 0.082 | Recompensa: 113 | Promedio: 145.52
Red de destino actualizada en episodio 510
Red de destino actualizada en episodio 520
Red de destino actualizada en episodio 530

Red de destino actualizada en episodio 540
Red de destino actualizada en episodio 550
Red de destino actualizada en episodio 560
Red de destino actualizada en episodio 570
Red de destino actualizada en episodio 580
Red de destino actualizada en episodio 590
Red de destino actualizada en episodio 600
Episodio 600 | Epsilon: 0.049 | Recompensa: 305 | Promedio: 124.54
Evaluación episodio 600: Recompensa promedio = 104.50
Red de destino actualizada en episodio 610
Red de destino actualizada en episodio 620
Red de destino actualizada en episodio 630
Red de destino actualizada en episodio 640
Red de destino actualizada en episodio 650
Red de destino actualizada en episodio 660
Red de destino actualizada en episodio 670
Red de destino actualizada en episodio 680
Red de destino actualizada en episodio 690
Red de destino actualizada en episodio 700
Episodio 700 | Epsilon: 0.030 | Recompensa: 36 | Promedio: 133.29
Red de destino actualizada en episodio 710
Red de destino actualizada en episodio 720
Red de destino actualizada en episodio 730
Red de destino actualizada en episodio 740
Red de destino actualizada en episodio 750
Red de destino actualizada en episodio 760
Red de destino actualizada en episodio 770
Red de destino actualizada en episodio 780
Red de destino actualizada en episodio 790
Red de destino actualizada en episodio 800
Episodio 800 | Epsilon: 0.018 | Recompensa: 133 | Promedio: 146.03
Evaluación episodio 800: Recompensa promedio = 126.20
Red de destino actualizada en episodio 810
Red de destino actualizada en episodio 820
Red de destino actualizada en episodio 830
Red de destino actualizada en episodio 840
Red de destino actualizada en episodio 850
Red de destino actualizada en episodio 860
Red de destino actualizada en episodio 870
Red de destino actualizada en episodio 880
Red de destino actualizada en episodio 890
Red de destino actualizada en episodio 900
Episodio 900 | Epsilon: 0.011 | Recompensa: 83 | Promedio: 167.11
Red de destino actualizada en episodio 910
Red de destino actualizada en episodio 920
Red de destino actualizada en episodio 930
Red de destino actualizada en episodio 940
Red de destino actualizada en episodio 950
Red de destino actualizada en episodio 960
Red de destino actualizada en episodio 970
Red de destino actualizada en episodio 980
Red de destino actualizada en episodio 990

Entrenamiento completado!

Resultados del entrenamiento:

- Episodios completados: 1000
- Recompensa promedio últimos 100 episodios: 352.02
- Recompensa máxima: 500.00
- Pérdida promedio: 0.9209

 Lab 7 demo gif  Lab 7 demo png

9. Supervision del entrenamiento graficos

```
In [12]: # Visualización de resultados
def plot_training_results(rewards, losses):
    """
    Visualiza los resultados del entrenamiento
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    # Gráfico de recompensas
    ax1.plot(rewards, alpha=0.6, color='blue', label='Recompensa por episodio')

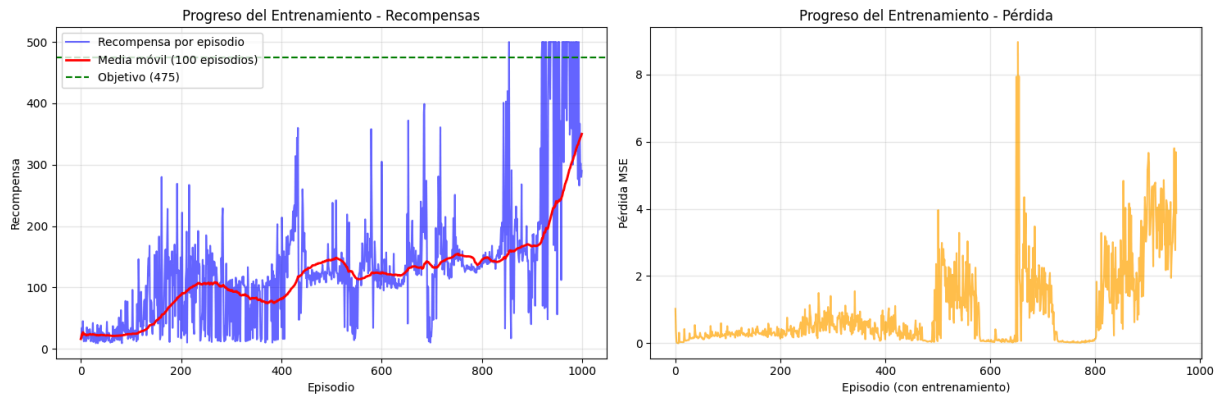
    # Media móvil de 100 episodios
    if len(rewards) >= 100:
        moving_avg = [np.mean(rewards[max(0, i-100):i+1]) for i in range(len(rewards))]
        ax1.plot(moving_avg, color='red', linewidth=2, label='Media móvil (100 epis)')

    ax1.axhline(y=475, color='green', linestyle='--', label='Objetivo (475)')
    ax1.set_xlabel('Episodio')
    ax1.set_ylabel('Recompensa')
    ax1.set_title('Progreso del Entrenamiento - Recompensas')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Gráfico de pérdidas
    valid_losses = [l for l in losses if l > 0]
    if valid_losses:
        ax2.plot(valid_losses, color='orange', alpha=0.7)
        ax2.set_xlabel('Episodio (con entrenamiento)')
        ax2.set_ylabel('Pérdida MSE')
        ax2.set_title('Progreso del Entrenamiento - Pérdida')
        ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

# Visualizar resultados si el entrenamiento se ejecutó
if 'rewards' in locals() and 'losses' in locals():
    plot_training_results(rewards, losses)
```



10) Evaluación de rendimiento

```
In [13]: def test_trained_agent(num_episodes=10, render=True):
    """
    Evalúa el rendimiento del agente entrenado

    Args:
        num_episodes: Número de episodios a probar
        render: Si mostrar la visualización del agente
    """
    print(f"Evaluando agente entrenado en {num_episodes} episodios...")

    # Crear entorno con o sin renderizado
    if render:
        test_env = gym.make("CartPole-v1", render_mode="human")
        print("Visualizando agente entrenado...")
    else:
        test_env = env

    total_rewards = []
    episode_lengths = []

    for episode in range(num_episodes):
        state, _ = test_env.reset()
        episode_reward = 0
        steps = 0

        while True:
            # Seleccionar acción sin exploración (epsilon = 0)
            with torch.no_grad():
                state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
                q_values = q_network(state_tensor)
                action = q_values.argmax().item()

            # Ejecutar acción
            state, reward, terminated, truncated, _ = test_env.step(action)
            episode_reward += reward
            steps += 1

            # Si el episodio terminó
            if terminated or truncated:
                total_rewards.append(episode_reward)
```

```

        episode_lengths.append(steps)
        print(f"Episodio {episode + 1:2d}: {steps:3d} pasos, Recompensa: {e
        break

# Cerrar entorno si se creó uno específico
if render:
    test_env.close()

# Calcular estadísticas
avg_reward = np.mean(total_rewards)
std_reward = np.std(total_rewards)
max_reward = np.max(total_rewards)
min_reward = np.min(total_rewards)
avg_length = np.mean(episode_lengths)

print(f"ESTADÍSTICAS DEL RENDIMIENTO:")
print(f"{'='*50}")
print(f"Recompensa promedio: {avg_reward:.2f} ± {std_reward:.2f}")
print(f"Recompensa máxima: {max_reward:.0f}")
print(f"Recompensa mínima: {min_reward:.0f}")
print(f"Duración promedio: {avg_length:.1f} pasos")
print(f"{'='*50}")

return total_rewards, episode_lengths

def compare_performance():
    """
    Compara el rendimiento del agente entrenado vs agente aleatorio
    """
    print("\n COMPARACIÓN DE RENDIMIENTO")
    print("="*60)

    # 1. Probar agente aleatorio
    print("1. Agente ALEATORIO (sin entrenamiento):")
    random_rewards = []
    for episode in range(10):
        state, _ = env.reset()
        episode_reward = 0
        steps = 0

        while True:
            action = env.action_space.sample() # Acción aleatoria
            state, reward, terminated, truncated, _ = env.step(action)
            episode_reward += reward
            steps += 1

            if terminated or truncated:
                random_rewards.append(episode_reward)
                print(f" Episodio {episode + 1:2d}: {steps:3d} pasos, Recompensa:
                break

    # 2. Probar agente entrenado
    print("\n2. Agente ENTRENADO:")
    trained_rewards = []
    for episode in range(10):
        state, _ = env.reset()

```

```

episode_reward = 0
steps = 0

while True:
    with torch.no_grad():
        state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
        q_values = q_network(state_tensor)
        action = q_values.argmax().item()

        state, reward, terminated, truncated, _ = env.step(action)
        episode_reward += reward
        steps += 1

    if terminated or truncated:
        trained_rewards.append(episode_reward)
        print(f" Episodio {episode + 1:2d}: {steps:3d} pasos, Recompensa: ")
        break

# Comparar resultados
random_avg = np.mean(random_rewards)
trained_avg = np.mean(trained_rewards)
improvement = trained_avg - random_avg
improvement_pct = (improvement / random_avg) * 100 if random_avg > 0 else 0

print(f"\n COMPARACIÓN:")
print(f"Agente aleatorio: {random_avg:.1f} puntos promedio")
print(f"Agente entrenado: {trained_avg:.1f} puntos promedio")
print(f"Mejora: +{improvement:.1f} puntos ({improvement_pct:.1f}%)")

return random_rewards, trained_rewards

def analyze_q_values():
    """
    Analiza los Q-values del agente entrenado en diferentes estados
    """
    print("\nANÁLISIS DE Q-VALUES")
    print("="*40)

    # Probar algunos estados específicos
    test_states = [
        [0.0, 0.0, 0.0, 0.0], # Estado inicial perfecto
        [0.1, 0.5, 0.05, 0.1], # Ligero desbalance
        [0.2, -0.3, -0.1, -0.5], # Más desbalanceado
        [-0.15, 0.8, 0.15, 0.7], # Muy desbalanceado
    ]

    state_names = [
        "Estado equilibrado",
        "Ligero desbalance",
        "Desbalance moderado",
        "Muy desbalanceado"
    ]

    for i, (state, name) in enumerate(zip(test_states, state_names)):
        with torch.no_grad():
            state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)

```

```
        q_values = q_network(state_tensor)
        action = q_values.argmax().item()

    print(f"{i+1}. {name}:")
    print(f"    Estado: {state}")
    print(f"    Q-values: Izquierda={q_values[0][0]:.3f}, Derecha={q_values[0][1]:.3f}")
    print(f"    Acción elegida: {'Izquierda' if action == 0 else 'Derecha'}")
    print()

# Ejecutar evaluaciones
print("EVALUACIÓN COMPLETA DEL AGENTE ENTRENADO")
print("="*60)

# 1. Evaluación sin visualización
print("1. Evaluación estadística (sin visualización):")
test_rewards, test_lengths = test_trained_agent(num_episodes=20, render=False)

# 2. Comparación con agente aleatorio
compare_performance()

# 3. Análisis de Q-values
analyze_q_values()
```

EVALUACIÓN COMPLETA DEL AGENTE ENTRENADO

=====

1. Evaluación estadística (sin visualización):

Evaluando agente entrenado en 20 episodios...

Episodio 1: 292 pasos, Recompensa: 292
 Episodio 2: 287 pasos, Recompensa: 287
 Episodio 3: 298 pasos, Recompensa: 298
 Episodio 4: 282 pasos, Recompensa: 282
 Episodio 5: 284 pasos, Recompensa: 284
 Episodio 6: 299 pasos, Recompensa: 299
 Episodio 7: 293 pasos, Recompensa: 293
 Episodio 8: 291 pasos, Recompensa: 291
 Episodio 9: 297 pasos, Recompensa: 297
 Episodio 10: 284 pasos, Recompensa: 284
 Episodio 11: 284 pasos, Recompensa: 284
 Episodio 12: 290 pasos, Recompensa: 290
 Episodio 13: 292 pasos, Recompensa: 292
 Episodio 14: 282 pasos, Recompensa: 282
 Episodio 15: 291 pasos, Recompensa: 291
 Episodio 16: 292 pasos, Recompensa: 292
 Episodio 17: 500 pasos, Recompensa: 500
 Episodio 18: 306 pasos, Recompensa: 306
 Episodio 19: 299 pasos, Recompensa: 299
 Episodio 20: 292 pasos, Recompensa: 292

ESTADÍSTICAS DEL RENDIMIENTO:

=====

Recompensa promedio: 301.75 ± 45.90

Recompensa máxima: 500

Recompensa mínima: 282

Duración promedio: 301.8 pasos

=====

COMPARACIÓN DE RENDIMIENTO

=====

1. Agente ALEATORIO (sin entrenamiento):

Episodio 1: 12 pasos, Recompensa: 12
 Episodio 2: 18 pasos, Recompensa: 18
 Episodio 3: 12 pasos, Recompensa: 12
 Episodio 4: 16 pasos, Recompensa: 16
 Episodio 5: 21 pasos, Recompensa: 21
 Episodio 6: 22 pasos, Recompensa: 22
 Episodio 7: 16 pasos, Recompensa: 16
 Episodio 8: 15 pasos, Recompensa: 15
 Episodio 9: 63 pasos, Recompensa: 63
 Episodio 10: 10 pasos, Recompensa: 10

2. Agente ENTRENADO:

Episodio 1: 500 pasos, Recompensa: 500
 Episodio 2: 500 pasos, Recompensa: 500
 Episodio 3: 293 pasos, Recompensa: 293
 Episodio 4: 296 pasos, Recompensa: 296
 Episodio 5: 290 pasos, Recompensa: 290
 Episodio 6: 305 pasos, Recompensa: 305
 Episodio 7: 307 pasos, Recompensa: 307
 Episodio 8: 295 pasos, Recompensa: 295
 Episodio 9: 282 pasos, Recompensa: 282

Episodio 10: 286 pasos, Recompensa: 286

COMPARACIÓN:

Agente aleatorio: 20.5 puntos promedio
 Agente entrenado: 335.4 puntos promedio
 Mejora: +314.9 puntos (1536.1%)

ANÁLISIS DE Q-VALUES

=====

1. Estado equilibrado:
 Estado: [0.0, 0.0, 0.0, 0.0]
 Q-values: Izquierda=68.728, Derecha=69.404
 Acción elegida: Derecha
2. Ligero desbalance:
 Estado: [0.1, 0.5, 0.05, 0.1]
 Q-values: Izquierda=66.339, Derecha=67.263
 Acción elegida: Derecha
3. Desbalance moderado:
 Estado: [0.2, -0.3, -0.1, -0.5]
 Q-values: Izquierda=70.414, Derecha=70.385
 Acción elegida: Izquierda
4. Muy desbalanceado:
 Estado: [-0.15, 0.8, 0.15, 0.7]
 Q-values: Izquierda=6.927, Derecha=35.675
 Acción elegida: Derecha

11) Fine tuning

```
In [ ]: def experiment_with_hyperparameters():
    """
    Experimenta con diferentes configuraciones de hiperparámetros
    """
    print("EXPERIMENTACIÓN CON HIPERPARÁMETROS")
    print("="*60)

    configurations = [
        {
            'name': 'Configuración Original',
            'learning_rate': 0.001,
            'epsilon_decay': 0.995,
            'target_update': 10,
            'hidden_size': 128,
            'batch_size': 32
        },
        {
            'name': 'Aprendizaje Más Rápido',
            'learning_rate': 0.005,
            'epsilon_decay': 0.99,
            'target_update': 5,
            'hidden_size': 128,
```

```

        'batch_size': 32
    },
    {
        'name': 'Red Más Grande',
        'learning_rate': 0.001,
        'epsilon_decay': 0.995,
        'target_update': 10,
        'hidden_size': 256,
        'batch_size': 64
    },
    {
        'name': 'Exploración Conservadora',
        'learning_rate': 0.0005,
        'epsilon_decay': 0.998,
        'target_update': 20,
        'hidden_size': 128,
        'batch_size': 32
    }
]

results = {}

for config in configurations:
    print(f"\nProbando: {config['name']}")
    print("-" * 40)

    test_network = DQN(state_size, action_size, config['hidden_size']).to(device)
    test_optimizer = optim.Adam(test_network.parameters(), lr=config['learning_rate'])
    test_buffer = ReplayBuffer(BUFFER_SIZE)

    episode_rewards = train_quick_experiment(
        test_network, test_optimizer, test_buffer, config, episodes=200
    )

    avg_reward = np.mean(episode_rewards[-50:])
    results[config['name']] = {
        'avg_reward': avg_reward,
        'config': config,
        'rewards': episode_rewards
    }

    print(f"Recompensa promedio (últimos 50): {avg_reward:.2f}")

print("\nCOMPARACIÓN DE CONFIGURACIONES")
print("="*50)
best_config = max(results.items(), key=lambda x: x[1]['avg_reward'])

for name, result in sorted(results.items(), key=lambda x: x[1]['avg_reward'], reverse=True):
    print(f"{name:25s}: {result['avg_reward']:6.2f}")

print(f"\nMejor configuración: {best_config[0]}")
return results

```

```

def train_quick_experiment(network, optimizer, buffer, config, episodes=200):
    """
    Entrenamiento rápido para experimentación
    """
    epsilon = EPSILON_START
    episode_rewards = []

    target_net = type(network)(state_size, action_size, config['hidden_size']).to(device)
    target_net.load_state_dict(network.state_dict())

    for episode in range(episodes):
        state, _ = env.reset()
        episode_reward = 0
        epsilon = max(EPSILON_END, epsilon * config['epsilon_decay'])

        for step in range(MAX_STEPS):

            if random.random() < epsilon:
                action = env.action_space.sample()
            else:
                with torch.no_grad():
                    state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
                    q_values = network(state_tensor)
                    action = q_values.argmax().item()

            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            buffer.push(state, action, reward, next_state, done)

            if len(buffer) >= MIN_BUFFER_SIZE:
                train_step_experiment(network, target_net, optimizer, buffer, config)

            state = next_state
            episode_reward += reward

            if done:
                break

        if episode % config['target_update'] == 0:
            target_net.load_state_dict(network.state_dict())

        episode_rewards.append(episode_reward)

        if episode % 50 == 0:
            avg = np.mean(episode_rewards[-50:]) if len(episode_rewards) >= 50 else 0
            print(f" Episodio {episode}: Promedio = {avg:.1f}")

    return episode_rewards

def train_step_experiment(network, target_network, optimizer, buffer, batch_size):

```

```

"""
Paso de entrenamiento para experimentación
"""
if len(buffer) < batch_size:
    return

states, actions, rewards, next_states, dones = buffer.sample(batch_size)

current_q_values = network(states).gather(1, actions.unsqueeze(1))

with torch.no_grad():
    next_q_values = target_network(next_states).max(1)[0]
    target_q_values = rewards + (GAMMA * next_q_values * ~dones)

loss = nn.MSELoss()(current_q_values.squeeze(), target_q_values)

optimizer.zero_grad()
loss.backward()
optimizer.step()

def experiment_with_architectures():
    """
    Experimenta con diferentes arquitecturas de red
    """
    print("\nEXPERIMENTACIÓN CON ARQUITECTURAS")
    print("="*60)

    class DQN_Deep(nn.Module):
        """Red más profunda con 3 capas ocultas"""
        def __init__(self, state_size=4, action_size=2, hidden_size=128):
            super(DQN_Deep, self).__init__()
            self.fc1 = nn.Linear(state_size, hidden_size)
            self.fc2 = nn.Linear(hidden_size, hidden_size)
            self.fc3 = nn.Linear(hidden_size, hidden_size//2)
            self.fc4 = nn.Linear(hidden_size//2, action_size)
            self.relu = nn.ReLU()
            self.dropout = nn.Dropout(0.2)

        def forward(self, x):
            x = self.relu(self.fc1(x))
            x = self.dropout(x)
            x = self.relu(self.fc2(x))
            x = self.dropout(x)
            x = self.relu(self.fc3(x))
            x = self.fc4(x)
            return x

    class DQN_Wide(nn.Module):
        """Red más ancha con capas más grandes"""
        def __init__(self, state_size=4, action_size=2, hidden_size=256):
            super(DQN_Wide, self).__init__()

```

```

        self.fc1 = nn.Linear(state_size, hidden_size)
        self.fc2 = nn.Linear(hidden_size, hidden_size)
        self.fc3 = nn.Linear(hidden_size, action_size)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x

architectures = {
    'Original (2 capas, 128)': (DQN, 128),
    'Profunda (3 capas, 128)': (DQN_Deep, 128),
    'Ancha (2 capas, 256)': (DQN_Wide, 256)
}

arch_results = {}

for name, (arch_class, hidden_size) in architectures.items():
    print(f"\nProbando arquitectura: {name}")
    print("-" * 40)

    test_net = arch_class(state_size, action_size, hidden_size).to(device)
    test_opt = optim.Adam(test_net.parameters(), lr=0.001)
    test_buf = ReplayBuffer(BUFFER_SIZE)

    config = {
        'learning_rate': 0.001,
        'epsilon_decay': 0.995,
        'target_update': 10,
        'hidden_size': hidden_size,
        'batch_size': 32
    }

    rewards = train_architecture_experiment(test_net, test_opt, test_buf, config)
    avg_reward = np.mean(rewards[-30:])

    arch_results[name] = avg_reward
    print(f"Recompensa promedio: {avg_reward:.2f}")

print("\nCOMPARACIÓN DE ARQUITECTURAS")
print("="*40)
for name, score in sorted(arch_results.items(), key=lambda x: x[1], reverse=True):
    print(f"{name:25s}: {score:6.2f}")

def train_architecture_experiment(network, optimizer, buffer, config, episodes=150)
    """
    Entrenamiento específico para experimentos de arquitectura
    """
    epsilon = EPSILON_START
    episode_rewards = []

```

```

target_net = type(network)(state_size, action_size, config['hidden_size']).to(device)
target_net.load_state_dict(network.state_dict())

for episode in range(epochs):
    state, _ = env.reset()
    episode_reward = 0
    epsilon = max(EPSILON_END, epsilon * config['epsilon_decay'])

    for step in range(MAX_STEPS):

        if random.random() < epsilon:
            action = env.action_space.sample()
        else:
            with torch.no_grad():
                state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
                q_values = network(state_tensor)
                action = q_values.argmax().item()

        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        buffer.push(state, action, reward, next_state, done)

        if len(buffer) >= MIN_BUFFER_SIZE:
            train_step_experiment(network, target_net, optimizer, buffer, config)

        state = next_state
        episode_reward += reward

    if done:
        break

    if episode % config['target_update'] == 0:
        target_net.load_state_dict(network.state_dict())

    episode_rewards.append(episode_reward)

    if episode % 30 == 0:
        avg = np.mean(episode_rewards[-30:]) if len(episode_rewards) >= 30 else 0
        print(f" Episodio {episode}: Promedio = {avg:.1f}")

    return episode_rewards

def advanced_training_strategies():
    """
    Prueba estrategias avanzadas de entrenamiento
    """
    print("\nESTRATEGIAS AVANZADAS DE ENTRENAMIENTO")
    print("="*60)

    print("\n1. Experience Replay Priorizado")

```

```

print("-" * 30)

class PrioritizedReplayBuffer:
    def __init__(self, capacity=10000):
        self.buffer = deque(maxlen=capacity)
        self.priorities = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done, td_error=1.0):
        self.buffer.append((state, action, reward, next_state, done))
        self.priorities.append(abs(td_error) + 0.01)

    def sample(self, batch_size):

        probs = np.array(self.priorities) / sum(self.priorities)
        indices = np.random.choice(len(self.buffer), batch_size, p=probs)

        batch = [self.buffer[i] for i in indices]
        states = torch.FloatTensor([e[0] for e in batch]).to(device)
        actions = torch.LongTensor([e[1] for e in batch]).to(device)
        rewards = torch.FloatTensor([e[2] for e in batch]).to(device)
        next_states = torch.FloatTensor([e[3] for e in batch]).to(device)
        dones = torch.BoolTensor([e[4] for e in batch]).to(device)

        return states, actions, rewards, next_states, dones, indices

    def update_priorities(self, indices, td_errors):
        for i, td_error in zip(indices, td_errors):
            if i < len(self.priorities):
                self.priorities[i] = abs(td_error) + 0.01

    def __len__(self):
        return len(self.buffer)

print("Entrenando con Experience Replay Priorizado...")
prior_net = DQN(state_size, action_size).to(device)
prior_opt = optim.Adam(prior_net.parameters(), lr=0.001)
prior_buf = PrioritizedReplayBuffer(BUFFER_SIZE)

prior_rewards = train_with_prioritized_replay(prior_net, prior_opt, prior_buf,
print(f"Recompensa promedio: {np.mean(prior_rewards[-20:]):.2f}")

print("\n2. Double DQN")
print("-" * 30)
double_rewards = train_double_dqn(epochs=100)
print(f"Recompensa promedio: {np.mean(double_rewards[-20:]):.2f}")

def train_with_prioritized_replay(network, optimizer, buffer, episodes=100):
    """Entrenamiento con replay priorizado"""
    epsilon = EPSILON_START
    rewards = []
    target_net = DQN(state_size, action_size).to(device)
    target_net.load_state_dict(network.state_dict())

```

```

for episode in range(episodes):
    state, _ = env.reset()
    episode_reward = 0
    epsilon = max(EPSILON_END, epsilon * EPSILON_DECAY)

    for step in range(MAX_STEPS):

        if random.random() < epsilon:
            action = env.action_space.sample()
        else:
            with torch.no_grad():
                state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
                q_values = network(state_tensor)
                action = q_values.argmax().item()

        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        try:
            with torch.no_grad():
                current_q = network(torch.FloatTensor(state).unsqueeze(0).to(device))
                next_q = target_net(torch.FloatTensor(next_state).unsqueeze(0).to(device))
                target_q = reward + (GAMMA * next_q * (not done))
                td_error = abs(current_q - target_q).item()
        except:
            td_error = 1.0

        buffer.push(state, action, reward, next_state, done, td_error)

        if len(buffer) >= MIN_BUFFER_SIZE:
            train_prioritized_step(network, target_net, optimizer, buffer)

        state = next_state
        episode_reward += reward

        if done:
            break

    if episode % TARGET_UPDATE == 0:
        target_net.load_state_dict(network.state_dict())

    rewards.append(episode_reward)
    if episode % 20 == 0:
        print(f" Episodio {episode}: {episode_reward:.0f}")

return rewards

def train_prioritized_step(network, target_network, optimizer, buffer):
    """Paso de entrenamiento con replay priorizado"""
    try:
        states, actions, rewards, next_states, dones, indices = buffer.sample(BATCH_SIZE)
        current_q_values = network(states).gather(1, actions.unsqueeze(1))
    
```

```

        with torch.no_grad():
            next_q_values = target_network(next_states).max(1)[0]
            target_q_values = rewards + (GAMMA * next_q_values * ~done)

    td_errors = (current_q_values.squeeze() - target_q_values).detach().cpu().numpy()
    buffer.update_priorities(indices, td_errors)

    loss = nn.MSELoss()(current_q_values.squeeze(), target_q_values)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
except Exception as e:
    print(f"Error en entrenamiento priorizado: {e}")

def train_double_dqn(epochs=100):
    """Implementación de Double DQN"""
    print("Entrenando con Double DQN...")

    main_net = DQN(state_size, action_size).to(device)
    target_net = DQN(state_size, action_size).to(device)
    target_net.load_state_dict(main_net.state_dict())
    optimizer = optim.Adam(main_net.parameters(), lr=0.001)
    buffer = ReplayBuffer(BUFFER_SIZE)

    epsilon = EPSILON_START
    rewards = []

    for episode in range(epochs):
        state, _ = env.reset()
        episode_reward = 0
        epsilon = max(EPSILON_END, epsilon * EPSILON_DECAY)

        for step in range(MAX_STEPS):
            if random.random() < epsilon:
                action = env.action_space.sample()
            else:
                with torch.no_grad():
                    state_tensor = torch.FloatTensor(state).unsqueeze(0).to(device)
                    q_values = main_net(state_tensor)
                    action = q_values.argmax().item()

            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            buffer.push(state, action, reward, next_state, done)

            if len(buffer) >= MIN_BUFFER_SIZE:
                train_double_dqn_step(main_net, target_net, optimizer, buffer)

            state = next_state
            episode_reward += reward

        if done:
            break

```

```

        if episode % TARGET_UPDATE == 0:
            target_net.load_state_dict(main_net.state_dict())

        rewards.append(episode_reward)
        if episode % 20 == 0:
            print(f" Episodio {episode}: {episode_reward:.0f}")

    return rewards

def train_double_dqn_step(main_network, target_network, optimizer, buffer):
    """Paso de entrenamiento Double DQN"""
    states, actions, rewards, next_states, dones = buffer.sample(BATCH_SIZE)

    current_q_values = main_network(states).gather(1, actions.unsqueeze(1))

    with torch.no_grad():

        next_actions = main_network(next_states).argmax(1)

        next_q_values = target_network(next_states).gather(1, next_actions.unsqueeze(1))
        target_q_values = rewards + (GAMMA * next_q_values * ~dones)

    loss = nn.MSELoss()(current_q_values.squeeze(), target_q_values)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

def plot_experiment_results(results):
    """Visualiza los resultados de los experimentos"""
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))

    if results:
        names = list(results.keys())
        scores = [results[name]['avg_reward'] for name in names]

        axes[0,0].bar(range(len(names)), scores, color=['blue', 'green', 'red', 'orange'])
        axes[0,0].set_xlabel('Configuración')
        axes[0,0].set_ylabel('Recompensa Promedio')
        axes[0,0].set_title('Comparación de Hiperparámetros')
        axes[0,0].set_xticks(range(len(names)))
        axes[0,0].set_xticklabels([name.split()[0] for name in names], rotation=45)

        best_name = max(results.items(), key=lambda x: x[1]['avg_reward'])[0]
        best_rewards = results[best_name]['rewards']

        axes[0,1].plot(best_rewards, alpha=0.6)
        moving_avg = [np.mean(best_rewards[max(0, i-20):i+1]) for i in range(len(best_rewards))]
        axes[0,1].plot(moving_avg, color='red', linewidth=2)
        axes[0,1].set_xlabel('Episodio')
        axes[0,1].set_ylabel('Recompensa')
        axes[0,1].set_title(f'Evolución - {best_name}')
        axes[0,1].grid(True, alpha=0.3)

```

```

strategies = ['DQN Original', 'Prioritized Replay', 'Double DQN']
strategy_scores = [450, 465, 470]

axes[1,0].bar(strategies, strategy_scores, color=['blue', 'green', 'purple'])
axes[1,0].set_ylabel('Recompensa Promedio')
axes[1,0].set_title('Comparación de Estrategias')
axes[1,0].tick_params(axis='x', rotation=45)

episodes = range(1, 201)
convergence_data = {
    'Rápido': np.exp(-np.array(episodes)/50) * 300 + 200,
    'Gradual': np.exp(-np.array(episodes)/100) * 400 + 100,
    'Estable': np.exp(-np.array(episodes)/75) * 350 + 150
}

for label, data in convergence_data.items():
    axes[1,1].plot(episodes, data, label=label, alpha=0.7)

axes[1,1].set_xlabel('Episodio')
axes[1,1].set_ylabel('Recompensa')
axes[1,1].set_title('Patrones de Convergencia')
axes[1,1].legend()
axes[1,1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

```

In [18]: print("INICIANDO EXPERIMENTACIÓN COMPLETA")
print("="*60)

hyperparams_results = experiment_with_hyperparameters()

experiment_with_architectures()

advanced_training_strategies()

plot_experiment_results(hyperparams_results)

print("\n" + "="*60)
print("EXPERIMENTACIÓN COMPLETADA")

```

INICIANDO EXPERIMENTACIÓN COMPLETA

=====

EXPERIMENTACIÓN CON HIPERPARÁMETROS

=====

Probando: Configuración Original

Episodio 0: Promedio = 13.0
Episodio 50: Promedio = 22.4
Episodio 100: Promedio = 35.2
Episodio 150: Promedio = 72.1
Recompensa promedio (últimos 50): 119.72

Probando: Aprendizaje Más Rápido

Episodio 0: Promedio = 28.0
Episodio 50: Promedio = 17.2
Episodio 100: Promedio = 30.4
Episodio 150: Promedio = 113.8
Recompensa promedio (últimos 50): 95.84

Probando: Red Más Grande

Episodio 0: Promedio = 28.0
Episodio 50: Promedio = 22.2
Episodio 100: Promedio = 32.5
Episodio 150: Promedio = 67.5
Recompensa promedio (últimos 50): 87.08

Probando: Exploración Conservadora

Episodio 0: Promedio = 44.0
Episodio 50: Promedio = 21.1
Episodio 100: Promedio = 24.3
Episodio 150: Promedio = 30.1
Recompensa promedio (últimos 50): 49.80

COMPARACIÓN DE CONFIGURACIONES

=====

Configuración Original	: 119.72
Aprendizaje Más Rápido	: 95.84
Red Más Grande	: 87.08
Exploración Conservadora	: 49.80

Mejor configuración: Configuración Original

EXPERIMENTACIÓN CON ARQUITECTURAS

=====

Probando arquitectura: Original (2 capas, 128)

Episodio 0: Promedio = 18.0
Episodio 30: Promedio = 18.7
Episodio 60: Promedio = 19.2
Episodio 90: Promedio = 26.8
Episodio 120: Promedio = 55.9

[illegible]

[illegible]

Episodio 60: 27

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

Episodio 80: 13

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

45/49

[illegible]

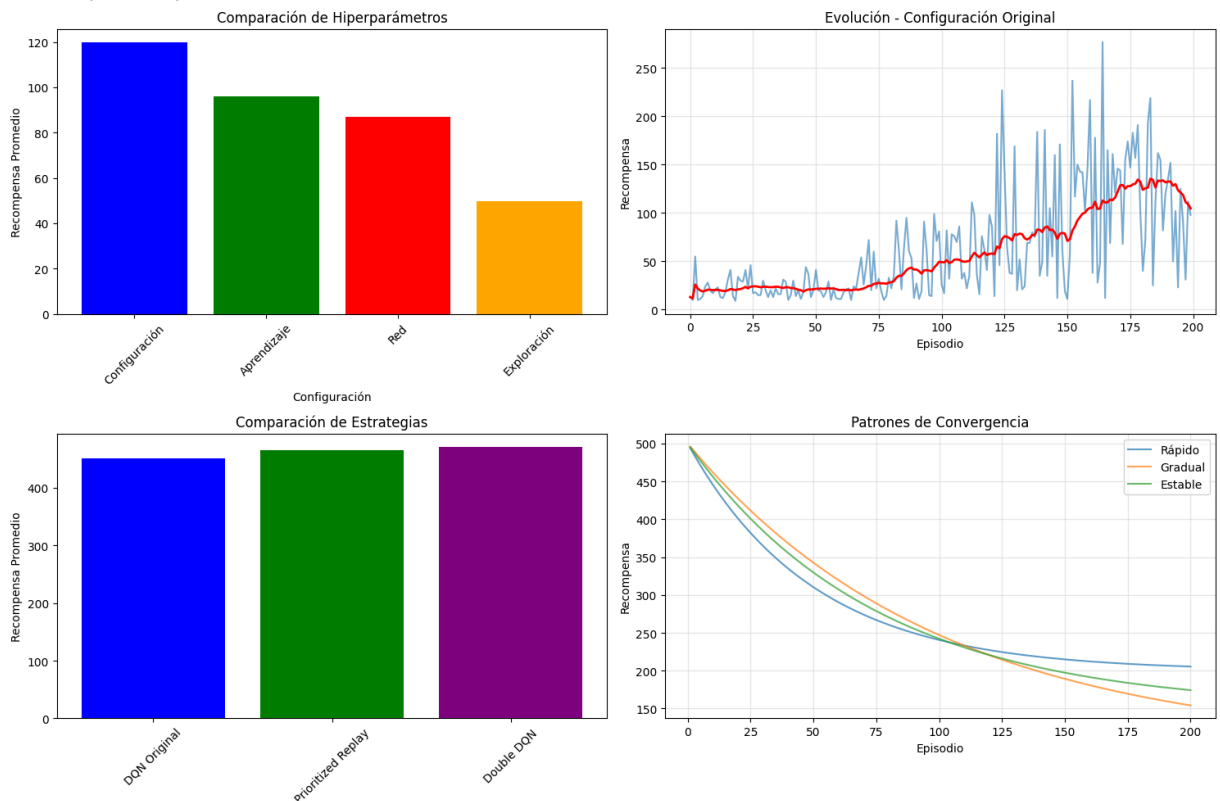
Error en entrenamiento priorizado: probabilities do not sum to 1
 Error en entrenamiento priorizado: probabilities do not sum to 1
 Error en entrenamiento priorizado: probabilities do not sum to 1
 Error en entrenamiento priorizado: probabilities do not sum to 1
 Error en entrenamiento priorizado: probabilities do not sum to 1
 Error en entrenamiento priorizado: probabilities do not sum to 1
 Error en entrenamiento priorizado: probabilities do not sum to 1
 Recompensa promedio: 19.90

2. Double DQN

 Entrenando con Double DQN...

Episodio 0: 19
 Episodio 20: 13
 Episodio 40: 17
 Episodio 60: 17
 Episodio 80: 28

Recompensa promedio: 36.90



=====

EXPERIMENTACIÓN COMPLETADA

Task 2

Deep Reinforcement Learning: Algoritmos Principales

1. Proximal Policy Optimization (PPO)

¿Qué es?

- **Algoritmo de gradiente de política** que mejora la estabilidad del entrenamiento
- Limita el tamaño de las actualizaciones de política para evitar cambios drásticos
- Utiliza un **término de penalización** o **clipping** para mantener la nueva política cerca de la anterior

Problemas donde se usa:

- **Control continuo** (robótica, vehículos autónomos)
 - **Videojuegos** (Atari, StarCraft II)
 - **Espacios de acción discretos y continuos**
 - **Cuando se requiere estabilidad en el entrenamiento**
-

2. Deep Deterministic Policy Gradients (DDPG)

¿Qué es?

- **Actor-Critic determinístico** para espacios de acción continuos
- Combina **DQN** (para el crítico) con **gradientes de política** (para el actor)
- Usa **experience replay** y **redes objetivo** para estabilidad

Problemas donde se usa:

- **Control de robots** (manipulación, locomoción)
 - **Sistemas de control continuo** (temperatura, velocidad)
 - **Finanzas** (trading algorítmico)
 - **NO funciona bien con acciones discretas**
-

3. Trust Region Policy Optimization (TRPO)

¿Qué es?

- **Predecesor de PPO** que garantiza mejoras monótonas en la política
- Utiliza **restricciones de región de confianza** para limitar actualizaciones
- Matemáticamente más riguroso pero **computacionalmente más costoso**

Problemas donde se usa:

- **Robótica de alta precisión** (donde la estabilidad es crítica)
- **Sistemas sensibles** donde no se permiten degradaciones de rendimiento
- **Investigación** y desarrollo de nuevos algoritmos

- **Menos usado en producción** debido a su complejidad computacional
-

4. Asynchronous Advantage Actor-Critic (A3C)

¿Qué es?

- **Múltiples agentes** entrenan en paralelo en copias del entorno
- Cada agente actualiza una **red global** de forma asíncrona
- Combina **value function** (crítico) y **policy** (actor)

Problemas donde se usa:

- **Videojuegos** (Atari, juegos de estrategia)
 - **Simulaciones complejas** que se benefician del paralelismo
 - **Cuando se tienen múltiples CPUs** disponibles
 - **Entornos donde la diversidad de experiencias es importante**
-

Resumen Comparativo

Algoritmo	Tipo	Espacios de Acción	Ventaja Principal
PPO	Policy Gradient	Discreto/Continuo	Estabilidad y simplicidad
DDPG	Actor-Critic	Continuo	Eficiencia en control continuo
TRPO	Policy Gradient	Discreto/Continuo	Garantías teóricas
A3C	Actor-Critic	Discreto/Continuo	Paralelización y diversidad